



Bilkent University

EEE 443 - Neural Networks

First Assignment Report

Fall 2025–2026

Aleyna Yaren Öztürken - 22103420

October 31, 2025

Contents

1	Introduction	1
2	Methodology	1
2.1	Data Generation and Classification	1
2.2	Perceptron Model	1
2.3	Training Algorithm	2
3	Experimental Setup	2
4	The Perceptron Algorithm Procedure	2
5	Results and Analysis	3
5.1	The original separating line for $n = 100$	3
5.2	The perceptron's final decision line for $n = 100$	3
5.3	Learning Rate $\eta = 0.1$ for $n = 100$	3
5.4	Learning Rate $\eta = 1$ for $n = 100$	4
5.5	Learning Rate $\eta = 10$ for $n = 100$	5
5.6	The original separating line for $n = 500$	5
5.7	The perceptron's final decision line for $n = 500$	5
5.8	Learning Rate $\eta = 0.1$ for $n = 500$	6
5.9	Learning Rate $\eta = 1$ for $n = 500$	6
5.10	Learning Rate $\eta = 10$ for $n = 500$	7
6	Overall Comments	8
7	Conclusion	10

1 Introduction

In this assignment, one of the simplest and earliest models of artificial neural networks which is, the single layer perceptron, implemented from scratch. The perceptron, originally proposed by Rosenblatt (1958), is a linear classifier that maps an input vector to a binary output by computing a weighted sum of the inputs and passing it through a threshold (activation) function [1]. In its basic form with two inputs, the model learns a linear decision boundary that separates two classes. Learning is achieved through an iterative weight-update rule that adjusts the weights in the direction that reduces classification errors [2].

2 Methodology

The main objective of this assignment is to implement and analyze the perceptron training algorithm for a two-class classification problem. The objective is to program a single perceptron from scratch, using only fundamental Python libraries like NumPy and Matplotlib, to learn a linear decision boundary. The study involves generating a linearly separable dataset, training the perceptron to correctly classify the data, and systematically analyzing the algorithm's performance. The key parameters investigated are the learning rate (η) and the number of data patterns (n). The results, including convergence behavior and the final learned decision boundary, are visualized and discussed for various configurations.

2.1 Data Generation and Classification

As it is stated in the assignment, for a given number of patterns n , a set of n points, $\mathbf{S} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, was generated by sampling uniformly from the unit square, i.e., $\mathbf{x}_i = (x_1, x_2)^T$ where $x_1, x_2 \in [0, 1]$. Note that for readability the symbol "S" is not used as a variable name in the code; however, it is added to comments for clarity. Specifically for this assignment n is required to be 100 and 500 to understand the experiment results more clearly.

After the generation of the data points, a random separating line was defined which passes through $(0, a)$ and $(1, b)$, where a, b independently sampled from $[0, 1]$. The original equation of this line is $x_2 = (b - a)x_1 + a$. Points were classified based on their position relative to this line:

- **Class 1 (desired output $d = 1$):** Points on or above the line, satisfying $(b - a)x_1 - x_2 + a \leq 0$.
- **Class 2 (desired output $d = 0$):** Points below the line, satisfying $(b - a)x_1 - x_2 + a > 0$.

2.2 Perceptron Model

A single perceptron with two inputs (x_1, x_2) , an adjustable threshold (θ), and two weights (w_1, w_2) was used. The activation potential v is calculated as:

$$v = w_1x_1 + w_2x_2 - \theta$$

The output y is determined by a step activation function:

$$y = f(v) = \begin{cases} 1 & \text{if } v \geq 0 \\ 0 & \text{if } v < 0 \end{cases}$$

For implementation, this is handled using an augmented weight vector $\mathbf{w} = [w_1, w_2, \theta]^T$ and an input vector $\mathbf{x} = [x_1, x_2, -1]^T$, such that $v = \mathbf{w}^T \mathbf{x}$.

2.3 Training Algorithm

In order to begin the training process, the initial weights w_1, w_2 and threshold θ were chosen randomly from $[-1, 1]$ as the assignment specified. The perceptron was trained by iterating through the dataset for multiple epochs. In each epoch, every point was evaluated according to the activation function. If a point was misclassified, the weights were updated according to the perceptron update rule:

$$\mathbf{w}(t+1) = \mathbf{w}(t) + \eta(d - y)\mathbf{x}$$

where η is the learning rate, d is the desired output, and y is the perceptron's actual output. The term $(d - y)$ evaluates to $+1$ or -1 for misclassifications and 0 otherwise, which correctly handles the weight updates for both classes. Since the data is linearly separable, the algorithm is guaranteed to converge. After the convergence (zero misclassifications in an epoch) training was stopped.

3 Experimental Setup

Since the data generation and separating line decision are random, to ensure getting the same results in each run and a fair comparison of the learning rates and dataset size from TA's perspective, a fixed random seed was used which is 41. This means that for a given n , the exact same set of points and the same original separating line were used for all three values of η (0.1, 1, and 10), isolating the learning rate as the sole variable.

4 The Perceptron Algorithm Procedure

The training process is an iterative algorithm designed to find a weight vector $\mathbf{w}$ that correctly classifies all points in the training set. The procedure is summarized in Algorithm 1.

Algorithm 1: The Perceptron Training Algorithm

Input: Training set $S = \{(\mathbf{x}_i, d_i)\}_{i=1}^n$. Learning rate $\eta > 0$.

Output: Final trained weight vector $\mathbf{w} = [w_1, w_2, \theta]^T$.

- **Initialize:** weight vector $\mathbf{w}$ with random values in $[-1, 1]$.
 - For each epoch until convergence:
 - Initialize $m \leftarrow 0$ which is the number of misclassified samples in the current epoch.
 - For each sample $(\mathbf{x}_i, d_i)$ in S :
 - * Predict output y for input $\mathbf{x}_i$ using current $\mathbf{w}$.
 - * If $y \neq d_i$:
 - Update weights: $\mathbf{w} \leftarrow \mathbf{w} + \eta(d_i - y)\mathbf{x}_i$.
 - Increment m by 1.
 - If $m = 0$, break the loop (convergence achieved).
 - Output final weight vector $\mathbf{w}$.
-

5 Results and Analysis

The simulation was first conducted with a dataset of 100 and then 500 points. The perceptron was trained using three different learning rates: $\eta = 0.1$, $\eta = 1$, and $\eta = 10$. Each configuration was run multiple times to observe the different effects on convergence. However as it is mentioned in the Experimental Setup section, a fixed random seed was used to ensure reproducibility and fair comparison. Each different learning rate converged in a different number of epochs. The results are summarized in Table 1 and Table 2. Furthermore the graphs showing the decision boundaries learned by the perceptron for each learning rate are presented in figures below with their corresponding misclassification vs epoch plots.

5.1 The original separating line for $n = 100$

The original decision boundary for $n = 100$ was is founded by the following equation: $x_2 = (b - a)x_1 + a$ where a and b are the y-intercepts at $x_1 = 0$ and $x_1 = 1$ respectively. For this specific run with random seed 41, the values of a and b were found to be approximately 0.6617 and -0.2377 respectively. Thus, the original separating line can be expressed as:

$$x_2 = 0.6617 + (-0.2377)x_1$$

The comparison between the original line and the final decision line of the perceptron can be seen in Table 1. It is evident that, for each learning rate, the final decision boundary of the perceptron closely matches the original line.

5.2 The perceptron's final decision line for $n = 100$

The final decision boundary line is determined by the weights learned by the perceptron after training. The equation of the decision boundary can be expressed in the form:

$$x_2 = \frac{\theta - w_1 x_1}{w_2}$$

where w_1 and w_2 are the weights associated with inputs x_1 and x_2 , and θ is the threshold. The specific equations for each learning rate after training are summarized in Table 1.

Learning Rate (η)	Epochs to Converge	Perceptron Decision Boundary
0.1	10	$x_2 = 0.6199 + x_1(-0.1810)$
1	9	$x_2 = 0.6487 + x_1(-0.1943)$
10	6	$x_2 = 0.6719 + x_1(-0.2480)$

Table 1: Convergence results for $n = 100$ with different learning rates.

5.3 Learning Rate $\eta = 0.1$ for $n = 100$

The figure below shows the data points with their class labels, along with the original and final perceptron decision boundaries on the left. The plot on the right displays the number of misclassified points versus the epoch number. Recall that the original separating line was $x_2 = 0.6617 + x_1(-0.2377)$. After 10 epochs for $\eta = 0.1$, the perceptron converged to $x_2 = 0.6199 + x_1(-0.1810)$.

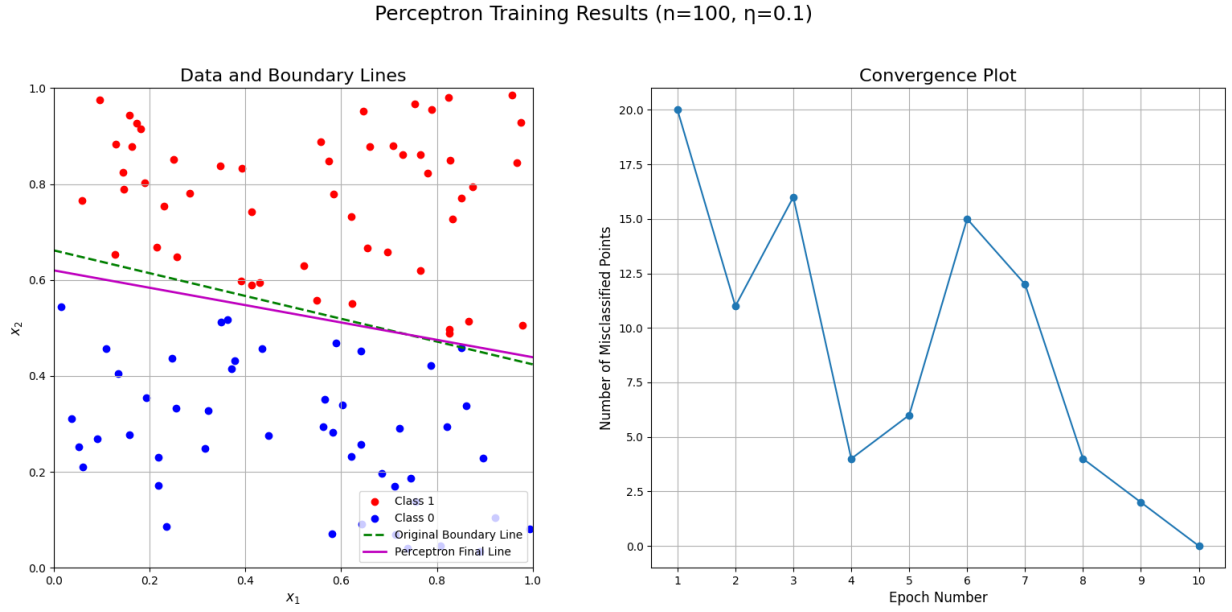


Figure 1: Data, boundary lines, and convergence plot for $n = 100, \eta = 0.1$.

From the figure above, it is clear that the learned line is very close to the original one, showing that the perceptron successfully captures the underlying decision boundary. The convergence plot shows fluctuations in the number of misclassified points. This happens because, while the algorithm aims to reduce misclassifications overall, correcting one point can occasionally cause another point to be misclassified. This is a common behavior in perceptron learning, which demonstrates that the convergence can not be achieved monotonically in general.

5.4 Learning Rate $\eta = 1$ for $n = 100$

The following figure shows the same plots for the learning rate $\eta = 1$. After 9 epochs for $\eta = 1$, the perceptron converged to $x_2 = 0.6487 + x_1(-0.1943)$.

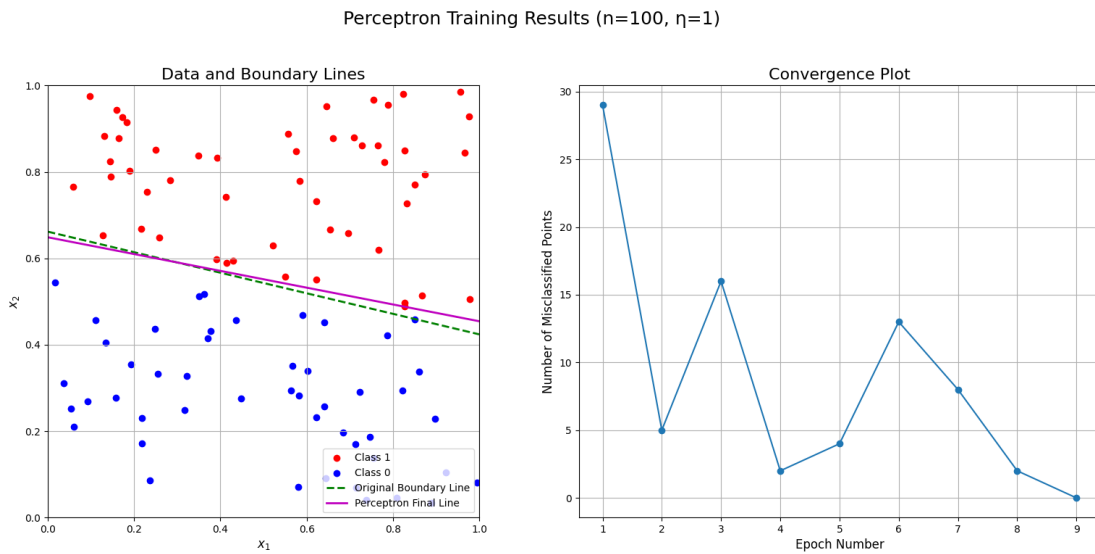


Figure 2: Data, boundary lines, and convergence plot for $n = 100, \eta = 1$.

Similar to the previous case, the learned decision boundary closely approximates the original line. The convergence plot again shows fluctuations in misclassifications, but with a larger learning rate, the changes are more significant. The perceptron makes larger adjustments to the weights, leading to quicker convergence but also more variability in misclassification counts.

5.5 Learning Rate $\eta = 10$ for $n = 100$

Lastly for $n = 100$, the following figure shows the same plots for the learning rate $\eta = 10$. After 6 epochs for $\eta = 10$, the perceptron converged to $x_2 = 0.6719 + x_1(-0.2480)$.

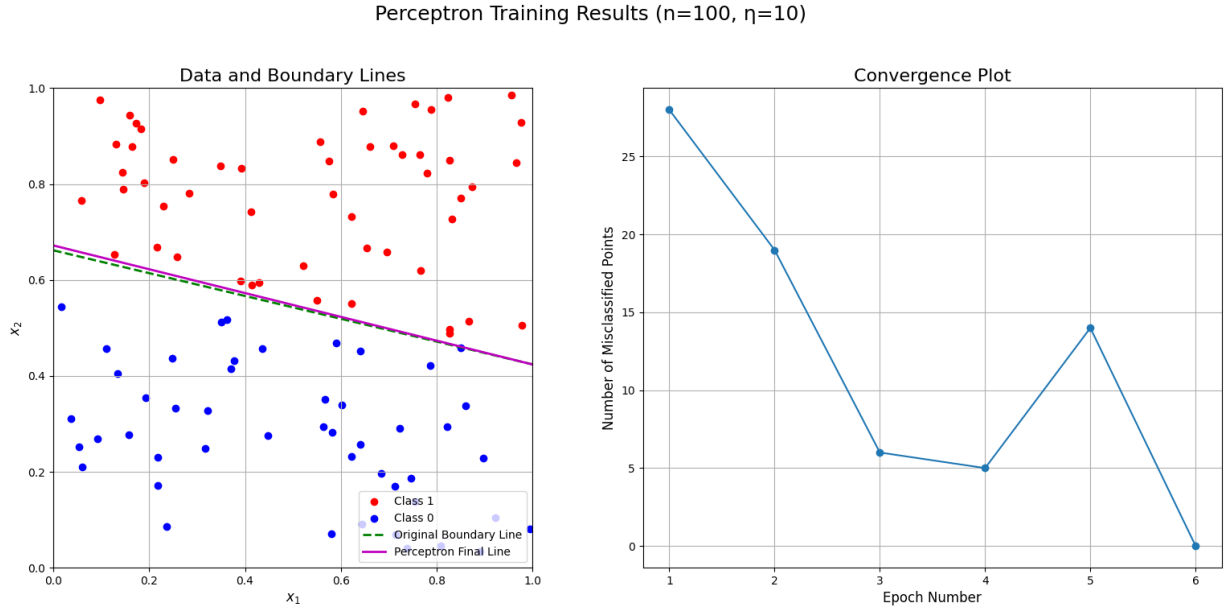


Figure 3: Data, boundary lines, and convergence plot for $n = 100, \eta = 10$.

This convergence plot shows that the perceptron reduces the number of misclassified points quite quickly overall, but not in a perfectly smooth way. Because the learning rate is relatively large ($\eta = 10$), each update changes the weights significantly, so the error sometimes goes back up after going down. This kind of fluctuation is typical when the step size is high: the algorithm is moving in roughly the right direction but occasionally overshoots the best boundary for the current epoch.

5.6 The original separating line for $n = 500$

Similar to the $n = 100$ case, the original decision boundary for $n = 500$ was calculated from the following equation: $x_2 = (b - a)x_1 + a$ where a and b are the y-intercepts at $x_1 = 0$ and $x_1 = 1$ respectively. Hence explicitly the original separating line can be expressed as:

$$x_2 = 0.7870 + (-0.2610)x_1$$

5.7 The perceptron's final decision line for $n = 500$

For each learning rate, the final decision boundary line is calculated from the following equation:

$$x_2 = \frac{\theta - w_1 x_1}{w_2}$$

where w_1 and w_2 are the weights associated with inputs x_1 and x_2 , and θ is the threshold. The specific equations for each learning rate after training are summarized in Table 2.

Learning Rate (η)	Epochs to Converge	Perceptron Decision Boundary
0.1	5	$x_2 = 0.7897 + x_1(-0.2623)$
1	7	$x_2 = 0.7845 + x_1(-0.2413)$
10	10	$x_2 = 0.7870 + x_1(-0.2514)$

Table 2: Convergence results for $n = 500$ with different learning rates.

5.8 Learning Rate $\eta = 0.1$ for $n = 500$

Left part of the figure illustrates the data points labeled by class together with both the original boundary and the final perceptron learned boundary. Right part of the figure plot shows how the number of misclassified samples changes across epochs. Recall that the original separating line was $x_2 = 0.7870 + x_1(-0.2610)$. After 5 epochs for $\eta = 0.1$, the perceptron converged to $x_2 = 0.7897 + x_1(-0.2623)$.



Figure 4: Data, boundary lines, and convergence plot for $n = 500, \eta = 0.1$.

The figure illustrates that the line learned by the perceptron almost coincides with the original one, indicating that the model has effectively learned the true decision boundary. The convergence curve shows smooth monotonic decline in misclassification points this is because of the random choice of initialization weights and small learning rate.

5.9 Learning Rate $\eta = 1$ for $n = 500$

The following figure shows the same plots for the learning rate $\eta = 1$. After 7 epochs, the perceptron converged to $x_2 = 0.7845 + x_1(-0.2413)$.

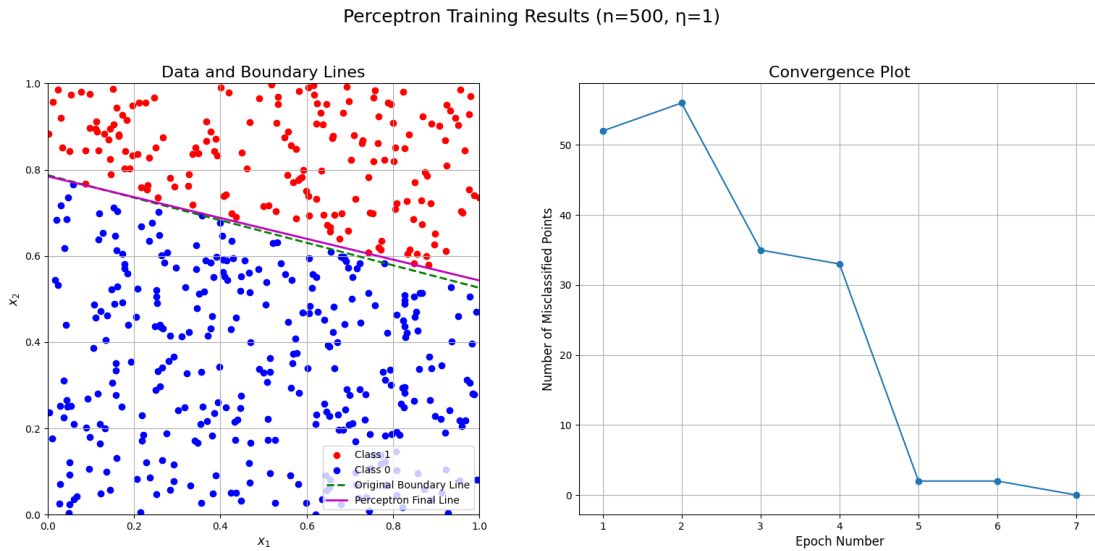


Figure 5: Data, boundary lines, and convergence plot for $n = 500, \eta = 1$.

The convergence plot shows a decreasing trend in the number of misclassified points, with some small fluctuations along the way. This pattern is typical for the perceptron. Updates may briefly increase error on a few samples which is expected in perceptron learning because an update that fixes some samples can temporarily worsen others. However the overall direction is toward fewer mistakes. Eventually the curve reaches zero misclassification, indicating successful convergence on this linearly separable dataset.

5.10 Learning Rate $\eta = 10$ for $n = 500$

Finally the following figure shows the same plots for the learning rate $\eta = 10$. After 10 epochs, the perceptron converged to $x_2 = 0.7870 + x_1(-0.2514)$.

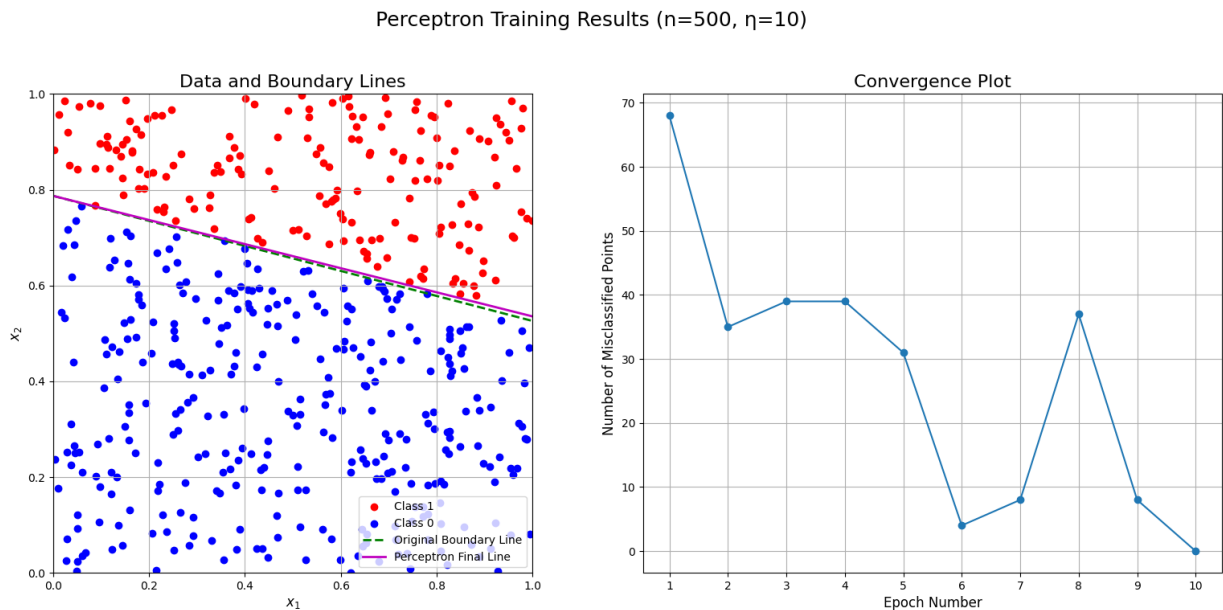


Figure 6: Data, boundary lines, and convergence plot for $n = 500, \eta = 10$.

This convergence plot is much less smooth than the previous one. With the larger learning rate ($\eta = 10$), the perceptron makes bigger weight updates, so the number of misclassified points fluctuates more and even rises again after dropping. This oscillatory behavior indicates that the updates sometimes overshoot the optimal boundary. Still, the overall trend is downward and the model eventually reaches zero misclassification, so convergence is achieved, just in a less stable way due to the high learning rate.

6 Overall Comments

The following two figures demonstrates the convergence plots for all learning rates for $n = 100$ and $n = 500$ respectively.

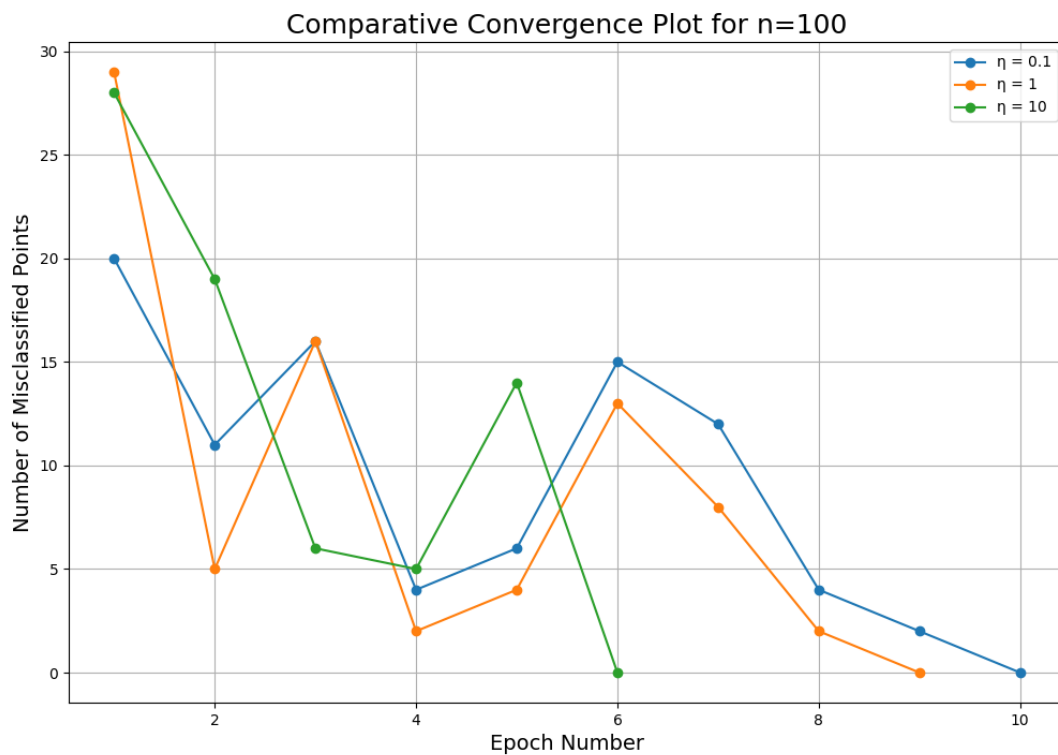


Figure 7: Convergence plots for all learning rates for $n = 100$.

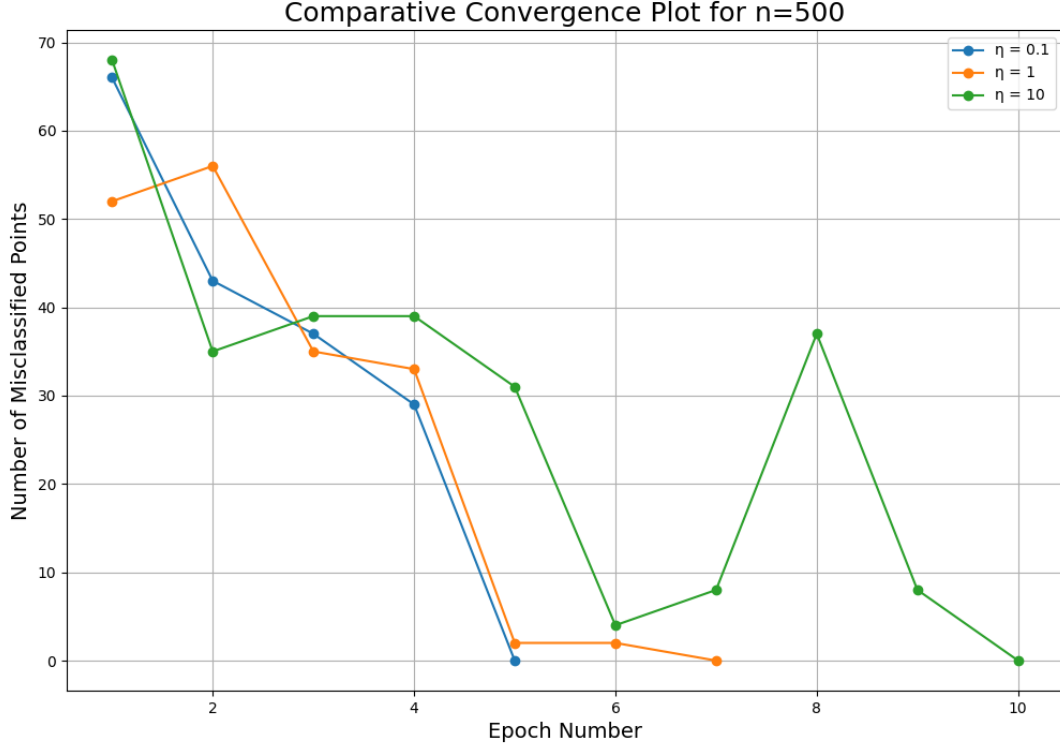


Figure 8: Convergence plots for all learning rates for $n = 500$.

Considering the both graphs together, one can not to be ensure that the large learning rate always leads to faster convergence. In fact, for $n = 500$, the largest learning rate $\eta = 10$ took the most epochs (10) to converge, while the smaller rates converged in fewer epochs (5 and 7). However one can observe that larger learning rates tend to produce more oscillatory convergence curves, indicating less stable learning.

Furthermore, since the perceptron is initialized with random weights and trained against a randomly generated target line, the training process does not always take the same number of epochs. In some runs it converged quickly, in others more slowly, and using a different learning rate sometimes led to faster convergence. These differences comes from the fact that the learning algorithm of perceptron and the update rule is dependendent on the random place of decision boundary, the particular sample distribution of the data in that run, and the randomly defined true separator. Nevertheless, in every experiment the algorithm eventually reduced the classification error to zero, which is consistent with the statement of the Perceptron Convergence Theorem [3]. Note that a single perceptron can only find a separating line, not necessarily the most optimal one. If an optimal or wider-margin separation is desired, methods such as Support Vector Machines (SVMs) are more suitable.

7 Conclusion

This assignment successfully demonstrated the implementation and behavior of the perceptron learning algorithm. The experiments confirmed that the algorithm reliably converges for linearly separable data, and that its performance is heavily influenced by the choices of learning rate and the size of the training set. The final learned decision boundaries, while mathematically different from the original line, were all valid solutions that correctly classified the datasets.

References

- [1] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] S. Haykin, *Neural Networks: A Comprehensive Foundation*. Upper Saddle River, NJ: Prentice Hall, 2 ed., 1999.
- [3] M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.